

Abstraction and Polymorphism

- 1.** `Payment.process(self, amount)` is overridden as `UPI.process(self, amount, pin)`. What is the best assessment?
- 2.** A parent declares `@abstractmethod def run()`. A child implements it, and a grandchild redeclares it as `@abstractmethod`. What happens?
- 3.** `B(A)` and `C(A)` each define `show()` that prints its own letter and then calls `super().show()`. `A.show()` prints "A". `D(B, C)` has no body. What does `D().show()` print?
- 4.** An abstract `calculate_fee()` is implemented by one child returning `float` and another returning `str`. What is the best fix?
- 5.** A developer says, "Abstraction means hiding everything, so I'll create one huge abstract class." What is the strongest challenge?
- 6.** A notification system supports Email, SMS, WhatsApp and Push. Which design best keeps business logic channel-agnostic?
- 7.** How do you let a document-processing system support a new file type without modifying existing processing logic?
- 8.** In a ride-booking system, `Bike/Car/Auto/PremiumCar` each calculate fares differently. Where does fare logic belong?
- 9.** A developer says if-elif on `payment_type` is simpler than polymorphism. When might they be right?
- 10.** Your team wants to replace the payment implementation without touching checkout. What boundary do you create?
- 11.** `Storage.save()` has children `MySQLStorage`, `MongoStorage` and `FileStorage`. Which is the best-defended choice?
- 12.** An ML app has `LinearRegression`, `DecisionTree`, `RandomForest` and `NeuralNetwork`, each with `train()` and `predict()`. How do you let the pipeline work with any model, including third-party ones?
- 13.** A `Payment.process(payment_type)` method branches on a string, and `UPIPayment/CardPayment` subclasses are then added. Why does this defeat polymorphism?
- 14.** A legacy codebase uses `type(obj) == X` and `isinstance(obj, X)` everywhere. What does this most likely indicate?

- 15.** A plugin system needs `initialize()`, `execute()` and `shutdown()` from unknown third parties. Which architecture works best?
- 16.** Why would an ABC deliberately mix concrete and abstract methods?
- 17.** With no declared interface, what decides whether an object can participate in polymorphism in Python?
- 18.** When is an ABC preferable to pure duck typing in production?
- 19.** What is the main problem with an ABC that has 15-20 abstract methods?
- 20.** What does `isinstance(obj, P)` check when `P` is a `@runtime_checkable` Protocol?
- 21.** After `Payment.register(LegacyPay)`, what is true?
- 22.** When does Python enforce unimplemented abstract methods?
- 23.** What is the correct way to declare an abstract property?
- 24.** Several mixins and a base class are combined with multiple inheritance. What is the safest `__init__` practice?
- 25.** What does `__init_subclass__` give a plugin system, and what is the catch?
- 26.** An `@abstractmethod` has a body. Is that meaningful?
- 27.** `Square` subclasses `Rectangle`, with width and height setters kept in sync. What's the issue?
- 28.** What is the key difference between `raise NotImplementedError` and `@abstractmethod`?
- 29.** An interface defines `fetch()`. Some implementations are `async def fetch()`, others are sync. What's the problem?
- 30.** When is `functools singledispatch` preferable to method-based polymorphism?